Open in Colab

Deep Learning for Function Estimation

We'll try to estimate the quality of red wine samples based on their physical and chemical features

We compute and report the mean absolute error, MAE

In [1]:

```
# CNN version
# Regression Example With wine dataset ...
import pandas as pd
from pandas import read_csv
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.layers import Dense , Dropout
from tensorflow.keras.layers import Conv1D
from tensorflow.keras.layers import MaxPooling1D
from tensorflow.keras.layers import Flatten
from tensorflow.keras.models import Sequential
from sklearn.model_selection import train_test_split
import numpy as np
import matplotlib.pyplot as plt
```

In [2]:

```
# mounting the Google drive
from google.colab import drive

drive.mount('/content/drive')

# Load the dataset
fp = "/content/drive/My Drive/Emeritus/Wi20260426/Module3-3/wine_r2_t_nohea
```

```
Mounted at /content/drive
```

In [3]:

```
# load dataset to a Pandas dataframe
dataframe = read_csv(fp , header=None)
# copy only values to a numpy n dimensional array, ndarray
dataset = dataframe.values

# split into input (X) and output (Y) variables
X = dataset[:,0:11]
Y = dataset[:,11]

print(X.shape)
X = X.reshape(X.shape[0], X.shape[1], 1)
print(X.shape)
```

```
(1599, 11)
(1599, 11, 1)
```

In [4]:

```
print(type(dataset))
print(dataset.shape)
print(dataframe.describe())
print( dataframe.head() )
```

```
<class 'numpy.ndarray'>
(1599, 12)
                0            1            2            3            4   \
count  1599.000000  1599.000000  1599.000000  1599.000000  1599.000000
mean      8.319637     0.527821     0.270976     2.538806     0.087467
std       1.741096     0.179060     0.194801     1.409928     0.047065
min       4.600000     0.120000     0.000000     0.900000     0.012000
25%       7.100000     0.390000     0.090000     1.900000     0.070000
50%       7.900000     0.520000     0.260000     2.200000     0.079000
75%       9.200000     0.640000     0.420000     2.600000     0.090000
max      15.900000     1.580000     1.000000    15.500000     0.611000

                5            6            7            8            9   \
count  1599.000000  1599.000000  1599.000000  1599.000000  1599.000000
mean     15.874922    46.467792     0.996747     3.311113     0.658149
std      10.460157    32.895324     0.001887     0.154386     0.169507
min       1.000000     6.000000     0.990070     2.740000     0.330000
25%       7.000000    22.000000     0.995600     3.210000     0.550000
50%      14.000000    38.000000     0.996750     3.310000     0.620000
75%      21.000000    62.000000     0.997835     3.400000     0.730000
max      72.000000   289.000000     1.003690     4.010000     2.000000

               10           11
count  1599.000000  1599.000000
mean     10.422983     5.636023
std       1.065668     0.807569
min       8.400000     3.000000
25%       9.500000     5.000000
50%      10.200000     6.000000
75%      11.100000     6.000000
max      14.900000     8.000000
     0     1     2    3      4     5     6       7     8     9    10  11
0   7.4  0.70  0.00  1.9  0.076  11.0  34.0  0.9978  3.51  0.56  9.4   5
1   7.8  0.88  0.00  2.6  0.098  25.0  67.0  0.9968  3.20  0.68  9.8   5
2   7.8  0.76  0.04  2.3  0.092  15.0  54.0  0.9970  3.26  0.65  9.8   5
3  11.2  0.28  0.56  1.9  0.075  17.0  60.0  0.9980  3.16  0.58  9.8   6
4   7.4  0.70  0.00  1.9  0.076  11.0  34.0  0.9978  3.51  0.56  9.4   5
```

In [5]:

```
# define base model, a deep covolutional one
# structure comprisses 2 1d convolution layers with 16 and 12 filters, filt
# then maxpooling for downsampling, a fully connected dense layer with 12 n
# Droput layer with 20% at the end
#

# //Winston20260426 eveApple2100h.ipynb ;
"""

def baseline_model():
    # create model
    model = Sequential()
    model.add(Conv1D(16, 3, activation="relu", input_shape=(11,1)))
    model.add(Conv1D(12, 3, activation="relu"))
    model.add(MaxPooling1D())
    model.add(Flatten())
    model.add(Dense(12, input_dim=8, kernel_initializer='normal', activatio
    model.add( Dropout(0.2) )
    model.add(Dense(1, kernel_initializer='normal'))
    # Compile model
    model.compile(loss='mean_squared_error', optimizer='adam')
    return model

"""
```

```
"""

def baseline_model():
    # create model
    model = Sequential()
    model.add(Conv1D(16, 3, activation="relu", input_shape=(11,1)))
    model.add(Conv1D(12, 3, activation="relu"))
    model.add(MaxPooling1D())
    model.add(Flatten())
    model.add(Dense(12, kernel_initializer='normal', activation='relu')) #
    model.add( Dropout(0.2) )
    model.add(Dense(1, kernel_initializer='normal'))
    # Compile model
    model.compile(loss='mean_squared_error', optimizer='adam')
    return model

"""



# //Winston20260426 py0006h.ipynb wi gemini solving input dimensions warnin

def baseline_model():
    # create model
    model = Sequential()
    model.add(Conv1D(16, 3, activation="relu", input_shape=(11,1)))
    model.add(Conv1D(12, 3, activation="relu"))
    model.add(MaxPooling1D())
    model.add(Flatten())
    model.add(Dense(12, kernel_initializer='normal', activation='relu')) #
    model.add( Dropout(0.2) )
    model.add(Dense(1, kernel_initializer='normal'))
    # Compile model
    model.compile(loss='mean_squared_error', optimizer='adam')
    return model
```

In [6]:

```
# splitting the train and test subsets
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, ra
```

In [7]:

```
# training the model for 900 epochs and testing the trained model with the
# we report the training error

# //Winston20260426 eveApple2100h.ipynb ;

estimator = baseline_model()
estimator.fit(X_train, Y_train, epochs=420, verbose=2)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_
conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument
to a layer. When using Sequential models, prefer using an `Input(shape)` obj
ect as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Epoch 1/420
40/40 - 6s - 147ms/step - loss: 31.2182
Epoch 2/420
40/40 - 0s - 3ms/step - loss: 17.0569
Epoch 3/420
40/40 - 0s - 3ms/step - loss: 11.5622
Epoch 4/420
40/40 - 0s - 3ms/step - loss: 10.6695
Epoch 5/420
40/40 - 0s - 3ms/step - loss: 8.8976
Epoch 6/420
40/40 - 0s - 3ms/step - loss: 6.2723
Epoch 7/420
40/40 - 0s - 3ms/step - loss: 4.8800
Epoch 8/420
40/40 - 0s - 3ms/step - loss: 4.6613
Epoch 9/420
40/40 - 0s - 3ms/step - loss: 4.2880
Epoch 10/420
40/40 - 0s - 3ms/step - loss: 4.5125
Epoch 11/420
40/40 - 0s - 3ms/step - loss: 4.3193
Epoch 12/420
40/40 - 0s - 3ms/step - loss: 4.3727
Epoch 13/420
40/40 - 0s - 3ms/step - loss: 4.6068
Epoch 14/420
40/40 - 0s - 3ms/step - loss: 4.3331
Epoch 15/420
40/40 - 0s - 3ms/step - loss: 4.4575
Epoch 16/420
40/40 - 0s - 3ms/step - loss: 4.2413
Epoch 17/420
40/40 - 0s - 3ms/step - loss: 4.4409
Epoch 18/420
40/40 - 0s - 3ms/step - loss: 3.8895
Epoch 19/420
40/40 - 0s - 3ms/step - loss: 3.9786
Epoch 20/420
40/40 - 0s - 3ms/step - loss: 4.1247
Epoch 21/420
40/40 - 0s - 3ms/step - loss: 4.2646
Epoch 22/420
40/40 - 0s - 3ms/step - loss: 3.8952
Epoch 23/420
40/40 - 0s - 3ms/step - loss: 3.8957
Epoch 24/420
40/40 - 0s - 3ms/step - loss: 4.0235
Epoch 25/420
40/40 - 0s - 3ms/step - loss: 3.8299
Epoch 26/420
40/40 - 0s - 3ms/step - loss: 4.1383
Epoch 27/420
40/40 - 0s - 3ms/step - loss: 3.6052
Epoch 28/420
40/40 - 0s - 3ms/step - loss: 4.0489
Epoch 29/420
40/40 - 0s - 4ms/step - loss: 3.7172
Epoch 30/420
40/40 - 0s - 5ms/step - loss: 3.9119
Epoch 31/420
40/40 - 0s - 8ms/step - loss: 3.6512
Epoch 32/420
```

```
40/40 - 0s - 5ms/step - loss: 3.5453
Epoch 33/420
40/40 - 0s - 5ms/step - loss: 3.7397
Epoch 34/420
40/40 - 0s - 5ms/step - loss: 3.8356
Epoch 35/420
40/40 - 0s - 4ms/step - loss: 3.9788
Epoch 36/420
40/40 - 0s - 3ms/step - loss: 3.4655
Epoch 37/420
40/40 - 0s - 3ms/step - loss: 3.3726
Epoch 38/420
40/40 - 0s - 3ms/step - loss: 3.1186
Epoch 39/420
40/40 - 0s - 3ms/step - loss: 3.2621
Epoch 40/420
40/40 - 0s - 3ms/step - loss: 3.3089
Epoch 41/420
40/40 - 0s - 3ms/step - loss: 3.7158
Epoch 42/420
40/40 - 0s - 7ms/step - loss: 3.1155
Epoch 43/420
40/40 - 0s - 3ms/step - loss: 3.3036
Epoch 44/420
40/40 - 0s - 5ms/step - loss: 3.3178
Epoch 45/420
40/40 - 0s - 4ms/step - loss: 3.2124
Epoch 46/420
40/40 - 0s - 4ms/step - loss: 3.2648
Epoch 47/420
40/40 - 0s - 8ms/step - loss: 3.2157
Epoch 48/420
40/40 - 0s - 7ms/step - loss: 3.3008
Epoch 49/420
40/40 - 0s - 4ms/step - loss: 3.0644
Epoch 50/420
40/40 - 0s - 8ms/step - loss: 3.0668
Epoch 51/420
40/40 - 0s - 7ms/step - loss: 3.1900
Epoch 52/420
40/40 - 0s - 4ms/step - loss: 3.2037
Epoch 53/420
40/40 - 0s - 4ms/step - loss: 3.2146
Epoch 54/420
40/40 - 0s - 4ms/step - loss: 3.0040
Epoch 55/420
40/40 - 0s - 5ms/step - loss: 3.0320
Epoch 56/420
40/40 - 0s - 5ms/step - loss: 3.0142
Epoch 57/420
40/40 - 0s - 7ms/step - loss: 2.8737
Epoch 58/420
40/40 - 0s - 5ms/step - loss: 2.9322
Epoch 59/420
40/40 - 0s - 4ms/step - loss: 2.8576
Epoch 60/420
40/40 - 0s - 3ms/step - loss: 2.7946
Epoch 61/420
40/40 - 0s - 3ms/step - loss: 2.9140
Epoch 62/420
40/40 - 0s - 4ms/step - loss: 2.7705
Epoch 63/420
40/40 - 0s - 3ms/step - loss: 2.8423
```

```
Epoch 64/420
40/40 - 0s - 3ms/step - loss: 2.7584
Epoch 65/420
40/40 - 0s - 3ms/step - loss: 2.6236
Epoch 66/420
40/40 - 0s - 3ms/step - loss: 2.6164
Epoch 67/420
40/40 - 0s - 3ms/step - loss: 2.6868
Epoch 68/420
40/40 - 0s - 3ms/step - loss: 2.7146
Epoch 69/420
40/40 - 0s - 3ms/step - loss: 2.4979
Epoch 70/420
40/40 - 0s - 3ms/step - loss: 2.6238
Epoch 71/420
40/40 - 0s - 3ms/step - loss: 2.8173
Epoch 72/420
40/40 - 0s - 3ms/step - loss: 2.5883
Epoch 73/420
40/40 - 0s - 3ms/step - loss: 2.5637
Epoch 74/420
40/40 - 0s - 3ms/step - loss: 2.4687
Epoch 75/420
40/40 - 0s - 3ms/step - loss: 2.4113
Epoch 76/420
40/40 - 0s - 3ms/step - loss: 2.7565
Epoch 77/420
40/40 - 0s - 3ms/step - loss: 2.1606
Epoch 78/420
40/40 - 0s - 3ms/step - loss: 2.3054
Epoch 79/420
40/40 - 0s - 3ms/step - loss: 2.3898
Epoch 80/420
40/40 - 0s - 3ms/step - loss: 2.1448
Epoch 81/420
40/40 - 0s - 3ms/step - loss: 2.3978
Epoch 82/420
40/40 - 0s - 3ms/step - loss: 2.3655
Epoch 83/420
40/40 - 0s - 3ms/step - loss: 2.2631
Epoch 84/420
40/40 - 0s - 3ms/step - loss: 2.3616
Epoch 85/420
40/40 - 0s - 3ms/step - loss: 2.3767
Epoch 86/420
40/40 - 0s - 3ms/step - loss: 2.2540
Epoch 87/420
40/40 - 0s - 3ms/step - loss: 2.1630
Epoch 88/420
40/40 - 0s - 3ms/step - loss: 2.2727
Epoch 89/420
40/40 - 0s - 3ms/step - loss: 2.1015
Epoch 90/420
40/40 - 0s - 3ms/step - loss: 2.0059
Epoch 91/420
40/40 - 0s - 3ms/step - loss: 2.1252
Epoch 92/420
40/40 - 0s - 3ms/step - loss: 2.1153
Epoch 93/420
40/40 - 0s - 3ms/step - loss: 2.2317
Epoch 94/420
40/40 - 0s - 3ms/step - loss: 2.0527
Epoch 95/420
```

```
40/40 - 0s - 3ms/step - loss: 2.1057
Epoch 96/420
40/40 - 0s - 3ms/step - loss: 1.8630
Epoch 97/420
40/40 - 0s - 3ms/step - loss: 1.9590
Epoch 98/420
40/40 - 0s - 4ms/step - loss: 1.9988
Epoch 99/420
40/40 - 0s - 3ms/step - loss: 1.8468
Epoch 100/420
40/40 - 0s - 3ms/step - loss: 1.9328
Epoch 101/420
40/40 - 0s - 3ms/step - loss: 1.9154
Epoch 102/420
40/40 - 0s - 3ms/step - loss: 1.7998
Epoch 103/420
40/40 - 0s - 3ms/step - loss: 1.7723
Epoch 104/420
40/40 - 0s - 8ms/step - loss: 1.8130
Epoch 105/420
40/40 - 0s - 11ms/step - loss: 1.8131
Epoch 106/420
40/40 - 0s - 6ms/step - loss: 1.7726
Epoch 107/420
40/40 - 0s - 6ms/step - loss: 1.6564
Epoch 108/420
40/40 - 0s - 3ms/step - loss: 1.6658
Epoch 109/420
40/40 - 0s - 3ms/step - loss: 1.6619
Epoch 110/420
40/40 - 0s - 3ms/step - loss: 1.7381
Epoch 111/420
40/40 - 0s - 7ms/step - loss: 1.7007
Epoch 112/420
40/40 - 0s - 4ms/step - loss: 1.6151
Epoch 113/420
40/40 - 0s - 3ms/step - loss: 1.6551
Epoch 114/420
40/40 - 0s - 3ms/step - loss: 1.6346
Epoch 115/420
40/40 - 0s - 3ms/step - loss: 1.6583
Epoch 116/420
40/40 - 0s - 3ms/step - loss: 1.6265
Epoch 117/420
40/40 - 0s - 3ms/step - loss: 1.5277
Epoch 118/420
40/40 - 0s - 3ms/step - loss: 1.5220
Epoch 119/420
40/40 - 0s - 3ms/step - loss: 1.6032
Epoch 120/420
40/40 - 0s - 3ms/step - loss: 1.6725
Epoch 121/420
40/40 - 0s - 3ms/step - loss: 1.6406
Epoch 122/420
40/40 - 0s - 3ms/step - loss: 1.6433
Epoch 123/420
40/40 - 0s - 3ms/step - loss: 1.5301
Epoch 124/420
40/40 - 0s - 3ms/step - loss: 1.4187
Epoch 125/420
40/40 - 0s - 3ms/step - loss: 1.3749
Epoch 126/420
40/40 - 0s - 4ms/step - loss: 1.4407
```

```
Epoch 127/420
40/40 - 0s - 3ms/step - loss: 1.4054
Epoch 128/420
40/40 - 0s - 3ms/step - loss: 1.2859
Epoch 129/420
40/40 - 0s - 5ms/step - loss: 1.4125
Epoch 130/420
40/40 - 0s - 5ms/step - loss: 1.4250
Epoch 131/420
40/40 - 0s - 4ms/step - loss: 1.4015
Epoch 132/420
40/40 - 0s - 4ms/step - loss: 1.2135
Epoch 133/420
40/40 - 0s - 4ms/step - loss: 1.4065
Epoch 134/420
40/40 - 0s - 4ms/step - loss: 1.3282
Epoch 135/420
40/40 - 0s - 4ms/step - loss: 1.3849
Epoch 136/420
40/40 - 0s - 7ms/step - loss: 1.2610
Epoch 137/420
40/40 - 0s - 4ms/step - loss: 1.3520
Epoch 138/420
40/40 - 0s - 8ms/step - loss: 1.2593
Epoch 139/420
40/40 - 0s - 8ms/step - loss: 1.1902
Epoch 140/420
40/40 - 0s - 4ms/step - loss: 1.1071
Epoch 141/420
40/40 - 0s - 5ms/step - loss: 1.2919
Epoch 142/420
40/40 - 0s - 5ms/step - loss: 1.2901
Epoch 143/420
40/40 - 0s - 5ms/step - loss: 1.2081
Epoch 144/420
40/40 - 0s - 5ms/step - loss: 1.1386
Epoch 145/420
40/40 - 0s - 5ms/step - loss: 1.1092
Epoch 146/420
40/40 - 0s - 6ms/step - loss: 1.1187
Epoch 147/420
40/40 - 0s - 3ms/step - loss: 1.1077
Epoch 148/420
40/40 - 0s - 4ms/step - loss: 1.1120
Epoch 149/420
40/40 - 0s - 3ms/step - loss: 1.0921
Epoch 150/420
40/40 - 0s - 3ms/step - loss: 1.0924
Epoch 151/420
40/40 - 0s - 3ms/step - loss: 1.1334
Epoch 152/420
40/40 - 0s - 3ms/step - loss: 1.1022
Epoch 153/420
40/40 - 0s - 3ms/step - loss: 1.0286
Epoch 154/420
40/40 - 0s - 3ms/step - loss: 1.0489
Epoch 155/420
40/40 - 0s - 3ms/step - loss: 1.0363
Epoch 156/420
40/40 - 0s - 4ms/step - loss: 1.0182
Epoch 157/420
40/40 - 0s - 3ms/step - loss: 1.0496
Epoch 158/420
```

```
40/40 - 0s - 3ms/step - loss: 1.0061
Epoch 159/420
40/40 - 0s - 4ms/step - loss: 1.0216
Epoch 160/420
40/40 - 0s - 3ms/step - loss: 0.9585
Epoch 161/420
40/40 - 0s - 3ms/step - loss: 0.9567
Epoch 162/420
40/40 - 0s - 3ms/step - loss: 1.0237
Epoch 163/420
40/40 - 0s - 3ms/step - loss: 0.9571
Epoch 164/420
40/40 - 0s - 3ms/step - loss: 0.9919
Epoch 165/420
40/40 - 0s - 4ms/step - loss: 0.9157
Epoch 166/420
40/40 - 0s - 3ms/step - loss: 0.9412
Epoch 167/420
40/40 - 0s - 3ms/step - loss: 0.9177
Epoch 168/420
40/40 - 0s - 3ms/step - loss: 0.8795
Epoch 169/420
40/40 - 0s - 3ms/step - loss: 0.9294
Epoch 170/420
40/40 - 0s - 3ms/step - loss: 0.8843
Epoch 171/420
40/40 - 0s - 3ms/step - loss: 0.8630
Epoch 172/420
40/40 - 0s - 3ms/step - loss: 0.8729
Epoch 173/420
40/40 - 0s - 3ms/step - loss: 0.8904
Epoch 174/420
40/40 - 0s - 3ms/step - loss: 0.8381
Epoch 175/420
40/40 - 0s - 3ms/step - loss: 0.8406
Epoch 176/420
40/40 - 0s - 3ms/step - loss: 0.8398
Epoch 177/420
40/40 - 0s - 4ms/step - loss: 0.8433
Epoch 178/420
40/40 - 0s - 3ms/step - loss: 0.8967
Epoch 179/420
40/40 - 0s - 4ms/step - loss: 0.8775
Epoch 180/420
40/40 - 0s - 3ms/step - loss: 0.8062
Epoch 181/420
40/40 - 0s - 3ms/step - loss: 0.8031
Epoch 182/420
40/40 - 0s - 3ms/step - loss: 0.7673
Epoch 183/420
40/40 - 0s - 3ms/step - loss: 0.8128
Epoch 184/420
40/40 - 0s - 3ms/step - loss: 0.7911
Epoch 185/420
40/40 - 0s - 3ms/step - loss: 0.7845
Epoch 186/420
40/40 - 0s - 3ms/step - loss: 0.7693
Epoch 187/420
40/40 - 0s - 3ms/step - loss: 0.7909
Epoch 188/420
40/40 - 0s - 3ms/step - loss: 0.7647
Epoch 189/420
40/40 - 0s - 3ms/step - loss: 0.7725
```

```
Epoch 190/420
40/40 - 0s - 3ms/step - loss: 0.7328
Epoch 191/420
40/40 - 0s - 3ms/step - loss: 0.7860
Epoch 192/420
40/40 - 0s - 3ms/step - loss: 0.7556
Epoch 193/420
40/40 - 0s - 3ms/step - loss: 0.7218
Epoch 194/420
40/40 - 0s - 3ms/step - loss: 0.7282
Epoch 195/420
40/40 - 0s - 3ms/step - loss: 0.7502
Epoch 196/420
40/40 - 0s - 3ms/step - loss: 0.6933
Epoch 197/420
40/40 - 0s - 3ms/step - loss: 0.7679
Epoch 198/420
40/40 - 0s - 3ms/step - loss: 0.6992
Epoch 199/420
40/40 - 0s - 3ms/step - loss: 0.7010
Epoch 200/420
40/40 - 0s - 3ms/step - loss: 0.7077
Epoch 201/420
40/40 - 0s - 3ms/step - loss: 0.7097
Epoch 202/420
40/40 - 0s - 3ms/step - loss: 0.6831
Epoch 203/420
40/40 - 0s - 3ms/step - loss: 0.6836
Epoch 204/420
40/40 - 0s - 3ms/step - loss: 0.6825
Epoch 205/420
40/40 - 0s - 3ms/step - loss: 0.6771
Epoch 206/420
40/40 - 0s - 3ms/step - loss: 0.6762
Epoch 207/420
40/40 - 0s - 3ms/step - loss: 0.6316
Epoch 208/420
40/40 - 0s - 3ms/step - loss: 0.6681
Epoch 209/420
40/40 - 0s - 3ms/step - loss: 0.6666
Epoch 210/420
40/40 - 0s - 3ms/step - loss: 0.6916
Epoch 211/420
40/40 - 0s - 3ms/step - loss: 0.6497
Epoch 212/420
40/40 - 0s - 3ms/step - loss: 0.6467
Epoch 213/420
40/40 - 0s - 3ms/step - loss: 0.6447
Epoch 214/420
40/40 - 0s - 3ms/step - loss: 0.6423
Epoch 215/420
40/40 - 0s - 3ms/step - loss: 0.6588
Epoch 216/420
40/40 - 0s - 3ms/step - loss: 0.6412
Epoch 217/420
40/40 - 0s - 3ms/step - loss: 0.6309
Epoch 218/420
40/40 - 0s - 3ms/step - loss: 0.6354
Epoch 219/420
40/40 - 0s - 3ms/step - loss: 0.6323
Epoch 220/420
40/40 - 0s - 3ms/step - loss: 0.6166
Epoch 221/420
```

```
40/40 - 0s - 3ms/step - loss: 0.6196
Epoch 222/420
40/40 - 0s - 5ms/step - loss: 0.6365
Epoch 223/420
40/40 - 0s - 4ms/step - loss: 0.6221
Epoch 224/420
40/40 - 0s - 4ms/step - loss: 0.6145
Epoch 225/420
40/40 - 0s - 4ms/step - loss: 0.6012
Epoch 226/420
40/40 - 0s - 5ms/step - loss: 0.6162
Epoch 227/420
40/40 - 0s - 4ms/step - loss: 0.6260
Epoch 228/420
40/40 - 0s - 4ms/step - loss: 0.5953
Epoch 229/420
40/40 - 0s - 4ms/step - loss: 0.6119
Epoch 230/420
40/40 - 0s - 4ms/step - loss: 0.6027
Epoch 231/420
40/40 - 0s - 8ms/step - loss: 0.5851
Epoch 232/420
40/40 - 0s - 6ms/step - loss: 0.5900
Epoch 233/420
40/40 - 0s - 4ms/step - loss: 0.5879
Epoch 234/420
40/40 - 0s - 4ms/step - loss: 0.5830
Epoch 235/420
40/40 - 0s - 8ms/step - loss: 0.5760
Epoch 236/420
40/40 - 0s - 5ms/step - loss: 0.5997
Epoch 237/420
40/40 - 0s - 5ms/step - loss: 0.5767
Epoch 238/420
40/40 - 0s - 5ms/step - loss: 0.5740
Epoch 239/420
40/40 - 0s - 7ms/step - loss: 0.6030
Epoch 240/420
40/40 - 0s - 3ms/step - loss: 0.5735
Epoch 241/420
40/40 - 0s - 3ms/step - loss: 0.5652
Epoch 242/420
40/40 - 0s - 3ms/step - loss: 0.5834
Epoch 243/420
40/40 - 0s - 3ms/step - loss: 0.5746
Epoch 244/420
40/40 - 0s - 3ms/step - loss: 0.5760
Epoch 245/420
40/40 - 0s - 3ms/step - loss: 0.5746
Epoch 246/420
40/40 - 0s - 3ms/step - loss: 0.5852
Epoch 247/420
40/40 - 0s - 3ms/step - loss: 0.5684
Epoch 248/420
40/40 - 0s - 3ms/step - loss: 0.5723
Epoch 249/420
40/40 - 0s - 3ms/step - loss: 0.5891
Epoch 250/420
40/40 - 0s - 4ms/step - loss: 0.5615
Epoch 251/420
40/40 - 0s - 3ms/step - loss: 0.5761
Epoch 252/420
40/40 - 0s - 3ms/step - loss: 0.5756
```

```
Epoch 253/420
40/40 - 0s - 3ms/step - loss: 0.5599
Epoch 254/420
40/40 - 0s - 3ms/step - loss: 0.5766
Epoch 255/420
40/40 - 0s - 3ms/step - loss: 0.5588
Epoch 256/420
40/40 - 0s - 3ms/step - loss: 0.5579
Epoch 257/420
40/40 - 0s - 3ms/step - loss: 0.5679
Epoch 258/420
40/40 - 0s - 4ms/step - loss: 0.5696
Epoch 259/420
40/40 - 0s - 3ms/step - loss: 0.5721
Epoch 260/420
40/40 - 0s - 3ms/step - loss: 0.5620
Epoch 261/420
40/40 - 0s - 3ms/step - loss: 0.5519
Epoch 262/420
40/40 - 0s - 3ms/step - loss: 0.5650
Epoch 263/420
40/40 - 0s - 3ms/step - loss: 0.5633
Epoch 264/420
40/40 - 0s - 3ms/step - loss: 0.5555
Epoch 265/420
40/40 - 0s - 3ms/step - loss: 0.5820
Epoch 266/420
40/40 - 0s - 3ms/step - loss: 0.5751
Epoch 267/420
40/40 - 0s - 3ms/step - loss: 0.5561
Epoch 268/420
40/40 - 0s - 3ms/step - loss: 0.5578
Epoch 269/420
40/40 - 0s - 3ms/step - loss: 0.5528
Epoch 270/420
40/40 - 0s - 3ms/step - loss: 0.5610
Epoch 271/420
40/40 - 0s - 3ms/step - loss: 0.5527
Epoch 272/420
40/40 - 0s - 3ms/step - loss: 0.5581
Epoch 273/420
40/40 - 0s - 3ms/step - loss: 0.5616
Epoch 274/420
40/40 - 0s - 3ms/step - loss: 0.5569
Epoch 275/420
40/40 - 0s - 3ms/step - loss: 0.5585
Epoch 276/420
40/40 - 0s - 3ms/step - loss: 0.5565
Epoch 277/420
40/40 - 0s - 3ms/step - loss: 0.5573
Epoch 278/420
40/40 - 0s - 3ms/step - loss: 0.5572
Epoch 279/420
40/40 - 0s - 3ms/step - loss: 0.5569
Epoch 280/420
40/40 - 0s - 3ms/step - loss: 0.5391
Epoch 281/420
40/40 - 0s - 3ms/step - loss: 0.5581
Epoch 282/420
40/40 - 0s - 3ms/step - loss: 0.5462
Epoch 283/420
40/40 - 0s - 3ms/step - loss: 0.5545
Epoch 284/420
```

```
40/40 - 0s - 3ms/step - loss: 0.5531
Epoch 285/420
40/40 - 0s - 3ms/step - loss: 0.5485
Epoch 286/420
40/40 - 0s - 3ms/step - loss: 0.5589
Epoch 287/420
40/40 - 0s - 3ms/step - loss: 0.5418
Epoch 288/420
40/40 - 0s - 3ms/step - loss: 0.5542
Epoch 289/420
40/40 - 0s - 3ms/step - loss: 0.5463
Epoch 290/420
40/40 - 0s - 3ms/step - loss: 0.5533
Epoch 291/420
40/40 - 0s - 3ms/step - loss: 0.5653
Epoch 292/420
40/40 - 0s - 3ms/step - loss: 0.5475
Epoch 293/420
40/40 - 0s - 3ms/step - loss: 0.5510
Epoch 294/420
40/40 - 0s - 3ms/step - loss: 0.5516
Epoch 295/420
40/40 - 0s - 3ms/step - loss: 0.5580
Epoch 296/420
40/40 - 0s - 3ms/step - loss: 0.5536
Epoch 297/420
40/40 - 0s - 3ms/step - loss: 0.5491
Epoch 298/420
40/40 - 0s - 3ms/step - loss: 0.5658
Epoch 299/420
40/40 - 0s - 3ms/step - loss: 0.5413
Epoch 300/420
40/40 - 0s - 3ms/step - loss: 0.5425
Epoch 301/420
40/40 - 0s - 4ms/step - loss: 0.5407
Epoch 302/420
40/40 - 0s - 3ms/step - loss: 0.5459
Epoch 303/420
40/40 - 0s - 3ms/step - loss: 0.5406
Epoch 304/420
40/40 - 0s - 3ms/step - loss: 0.5458
Epoch 305/420
40/40 - 0s - 3ms/step - loss: 0.5405
Epoch 306/420
40/40 - 0s - 4ms/step - loss: 0.5513
Epoch 307/420
40/40 - 0s - 3ms/step - loss: 0.5326
Epoch 308/420
40/40 - 0s - 3ms/step - loss: 0.5210
Epoch 309/420
40/40 - 0s - 3ms/step - loss: 0.5276
Epoch 310/420
40/40 - 0s - 3ms/step - loss: 0.5533
Epoch 311/420
40/40 - 0s - 3ms/step - loss: 0.5508
Epoch 312/420
40/40 - 0s - 4ms/step - loss: 0.5535
Epoch 313/420
40/40 - 0s - 3ms/step - loss: 0.5507
Epoch 314/420
40/40 - 0s - 3ms/step - loss: 0.5299
Epoch 315/420
40/40 - 0s - 4ms/step - loss: 0.5371
```

```
Epoch 316/420
40/40 - 0s - 5ms/step - loss: 0.5459
Epoch 317/420
40/40 - 0s - 5ms/step - loss: 0.5237
Epoch 318/420
40/40 - 0s - 4ms/step - loss: 0.5376
Epoch 319/420
40/40 - 0s - 5ms/step - loss: 0.5291
Epoch 320/420
40/40 - 0s - 7ms/step - loss: 0.5219
Epoch 321/420
40/40 - 0s - 4ms/step - loss: 0.5205
Epoch 322/420
40/40 - 0s - 4ms/step - loss: 0.5281
Epoch 323/420
40/40 - 0s - 4ms/step - loss: 0.5359
Epoch 324/420
40/40 - 0s - 7ms/step - loss: 0.5330
Epoch 325/420
40/40 - 0s - 8ms/step - loss: 0.5302
Epoch 326/420
40/40 - 0s - 4ms/step - loss: 0.5244
Epoch 327/420
40/40 - 0s - 8ms/step - loss: 0.5281
Epoch 328/420
40/40 - 0s - 5ms/step - loss: 0.5280
Epoch 329/420
40/40 - 0s - 8ms/step - loss: 0.5109
Epoch 330/420
40/40 - 0s - 4ms/step - loss: 0.5249
Epoch 331/420
40/40 - 0s - 3ms/step - loss: 0.5265
Epoch 332/420
40/40 - 0s - 3ms/step - loss: 0.5241
Epoch 333/420
40/40 - 0s - 4ms/step - loss: 0.5291
Epoch 334/420
40/40 - 0s - 3ms/step - loss: 0.5084
Epoch 335/420
40/40 - 0s - 3ms/step - loss: 0.5272
Epoch 336/420
40/40 - 0s - 3ms/step - loss: 0.5255
Epoch 337/420
40/40 - 0s - 3ms/step - loss: 0.5216
Epoch 338/420
40/40 - 0s - 3ms/step - loss: 0.5154
Epoch 339/420
40/40 - 0s - 3ms/step - loss: 0.5248
Epoch 340/420
40/40 - 0s - 3ms/step - loss: 0.5342
Epoch 341/420
40/40 - 0s - 4ms/step - loss: 0.5150
Epoch 342/420
40/40 - 0s - 3ms/step - loss: 0.5113
Epoch 343/420
40/40 - 0s - 3ms/step - loss: 0.5223
Epoch 344/420
40/40 - 0s - 3ms/step - loss: 0.5386
Epoch 345/420
40/40 - 0s - 3ms/step - loss: 0.5205
Epoch 346/420
40/40 - 0s - 3ms/step - loss: 0.5186
Epoch 347/420
```

```
40/40 - 0s - 3ms/step - loss: 0.5241
Epoch 348/420
40/40 - 0s - 3ms/step - loss: 0.5236
Epoch 349/420
40/40 - 0s - 3ms/step - loss: 0.5273
Epoch 350/420
40/40 - 0s - 3ms/step - loss: 0.5252
Epoch 351/420
40/40 - 0s - 3ms/step - loss: 0.5079
Epoch 352/420
40/40 - 0s - 3ms/step - loss: 0.5145
Epoch 353/420
40/40 - 0s - 3ms/step - loss: 0.5200
Epoch 354/420
40/40 - 0s - 3ms/step - loss: 0.5174
Epoch 355/420
40/40 - 0s - 3ms/step - loss: 0.5089
Epoch 356/420
40/40 - 0s - 3ms/step - loss: 0.5194
Epoch 357/420
40/40 - 0s - 3ms/step - loss: 0.5063
Epoch 358/420
40/40 - 0s - 3ms/step - loss: 0.5063
Epoch 359/420
40/40 - 0s - 3ms/step - loss: 0.5216
Epoch 360/420
40/40 - 0s - 3ms/step - loss: 0.5178
Epoch 361/420
40/40 - 0s - 3ms/step - loss: 0.5234
Epoch 362/420
40/40 - 0s - 3ms/step - loss: 0.5291
Epoch 363/420
40/40 - 0s - 3ms/step - loss: 0.5100
Epoch 364/420
40/40 - 0s - 3ms/step - loss: 0.5265
Epoch 365/420
40/40 - 0s - 4ms/step - loss: 0.5106
Epoch 366/420
40/40 - 0s - 3ms/step - loss: 0.5245
Epoch 367/420
40/40 - 0s - 3ms/step - loss: 0.5171
Epoch 368/420
40/40 - 0s - 3ms/step - loss: 0.5007
Epoch 369/420
40/40 - 0s - 3ms/step - loss: 0.5029
Epoch 370/420
40/40 - 0s - 3ms/step - loss: 0.4938
Epoch 371/420
40/40 - 0s - 3ms/step - loss: 0.5011
Epoch 372/420
40/40 - 0s - 3ms/step - loss: 0.5128
Epoch 373/420
40/40 - 0s - 4ms/step - loss: 0.5202
Epoch 374/420
40/40 - 0s - 7ms/step - loss: 0.5186
Epoch 375/420
40/40 - 0s - 3ms/step - loss: 0.5229
Epoch 376/420
40/40 - 0s - 3ms/step - loss: 0.5106
Epoch 377/420
40/40 - 0s - 3ms/step - loss: 0.5044
Epoch 378/420
40/40 - 0s - 3ms/step - loss: 0.5183
```

```
Epoch 379/420
40/40 - 0s - 4ms/step - loss: 0.5174
Epoch 380/420
40/40 - 0s - 3ms/step - loss: 0.5195
Epoch 381/420
40/40 - 0s - 3ms/step - loss: 0.5047
Epoch 382/420
40/40 - 0s - 3ms/step - loss: 0.5043
Epoch 383/420
40/40 - 0s - 3ms/step - loss: 0.4922
Epoch 384/420
40/40 - 0s - 3ms/step - loss: 0.4958
Epoch 385/420
40/40 - 0s - 3ms/step - loss: 0.5038
Epoch 386/420
40/40 - 0s - 3ms/step - loss: 0.4879
Epoch 387/420
40/40 - 0s - 4ms/step - loss: 0.4959
Epoch 388/420
40/40 - 0s - 3ms/step - loss: 0.5202
Epoch 389/420
40/40 - 0s - 3ms/step - loss: 0.5149
Epoch 390/420
40/40 - 0s - 3ms/step - loss: 0.5126
Epoch 391/420
40/40 - 0s - 3ms/step - loss: 0.5027
Epoch 392/420
40/40 - 0s - 3ms/step - loss: 0.4869
Epoch 393/420
40/40 - 0s - 3ms/step - loss: 0.4957
Epoch 394/420
40/40 - 0s - 3ms/step - loss: 0.5067
Epoch 395/420
40/40 - 0s - 4ms/step - loss: 0.5062
Epoch 396/420
40/40 - 0s - 3ms/step - loss: 0.4890
Epoch 397/420
40/40 - 0s - 3ms/step - loss: 0.5096
Epoch 398/420
40/40 - 0s - 3ms/step - loss: 0.4926
Epoch 399/420
40/40 - 0s - 3ms/step - loss: 0.4927
Epoch 400/420
40/40 - 0s - 3ms/step - loss: 0.4856
Epoch 401/420
40/40 - 0s - 3ms/step - loss: 0.5207
Epoch 402/420
40/40 - 0s - 3ms/step - loss: 0.5006
Epoch 403/420
40/40 - 0s - 4ms/step - loss: 0.5126
Epoch 404/420
40/40 - 0s - 3ms/step - loss: 0.4955
Epoch 405/420
40/40 - 0s - 4ms/step - loss: 0.4843
Epoch 406/420
40/40 - 0s - 4ms/step - loss: 0.4983
Epoch 407/420
40/40 - 0s - 5ms/step - loss: 0.4984
Epoch 408/420
40/40 - 0s - 4ms/step - loss: 0.5057
Epoch 409/420
40/40 - 0s - 5ms/step - loss: 0.4865
Epoch 410/420
```

```
40/40 - 0s - 4ms/step - loss: 0.5029
Epoch 411/420
40/40 - 0s - 8ms/step - loss: 0.4958
Epoch 412/420
40/40 - 0s - 5ms/step - loss: 0.5004
Epoch 413/420
40/40 - 0s - 4ms/step - loss: 0.4858
Epoch 414/420
40/40 - 0s - 4ms/step - loss: 0.4917
Epoch 415/420
40/40 - 0s - 7ms/step - loss: 0.4864
Epoch 416/420
40/40 - 0s - 4ms/step - loss: 0.4990
Epoch 417/420
40/40 - 0s - 4ms/step - loss: 0.5006
Epoch 418/420
40/40 - 0s - 4ms/step - loss: 0.4839
Epoch 419/420
40/40 - 0s - 8ms/step - loss: 0.4883
Epoch 420/420
40/40 - 0s - 7ms/step - loss: 0.5028
```

Out[7]:

```
<keras.src.callbacks.history.History at 0x7e19daa5b2c0>
```

In [7]:

```

```

In [8]:

```
# //Winston20260426 eveApple2100h.ipynb ;
# splitted this section off so we don't hv to run all the fit epochs again

prediction = estimator.predict(X_train)

train_error =  np.abs(Y_train - prediction)
mean_error = np.mean(train_error)
min_error = np.min(train_error)
max_error = np.max(train_error)
std_error = np.std(train_error)
print('\nTraining results:\nmean=',mean_error,'\nmin=',min_error,'\nmax=',m
```

```
40/40 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step

Training results:
mean= 0.735462718036909
min= 0.00033855438232421875
max= 4.021273612976074
std= 0.525753720707854
```

In [9]:

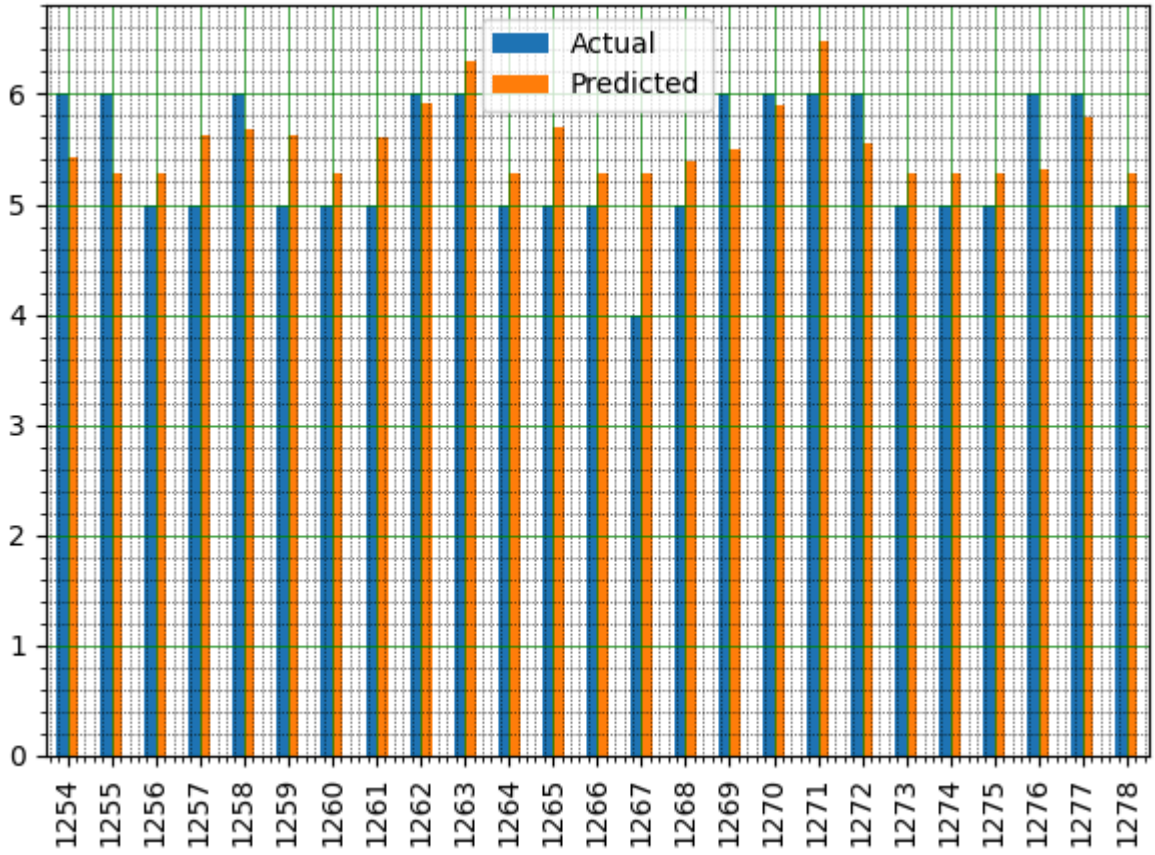
```
# showing the real and predicted quality scores for the training data
df_train1 = pd.DataFrame({'Actual': Y_train, 'Predicted': prediction.ravel(
df_train25 = df_train1.tail(25)
print(df_train25)
```

```
      Actual  Predicted
1254     6.0   5.423900
1255     6.0   5.275720
1256     5.0   5.275720
1257     5.0   5.622785
1258     6.0   5.678504
1259     5.0   5.618704
1260     5.0   5.275720
1261     5.0   5.605243
1262     6.0   5.916135
1263     6.0   6.298814
1264     5.0   5.275720
1265     5.0   5.692049
1266     5.0   5.275720
1267     4.0   5.275720
1268     5.0   5.388345
1269     6.0   5.507119
1270     6.0   5.894777
1271     6.0   6.482347
1272     6.0   5.550299
1273     5.0   5.275720
1274     5.0   5.275720
1275     5.0   5.275720
1276     6.0   5.322738
1277     6.0   5.786094
1278     5.0   5.275720
```

In [10]:

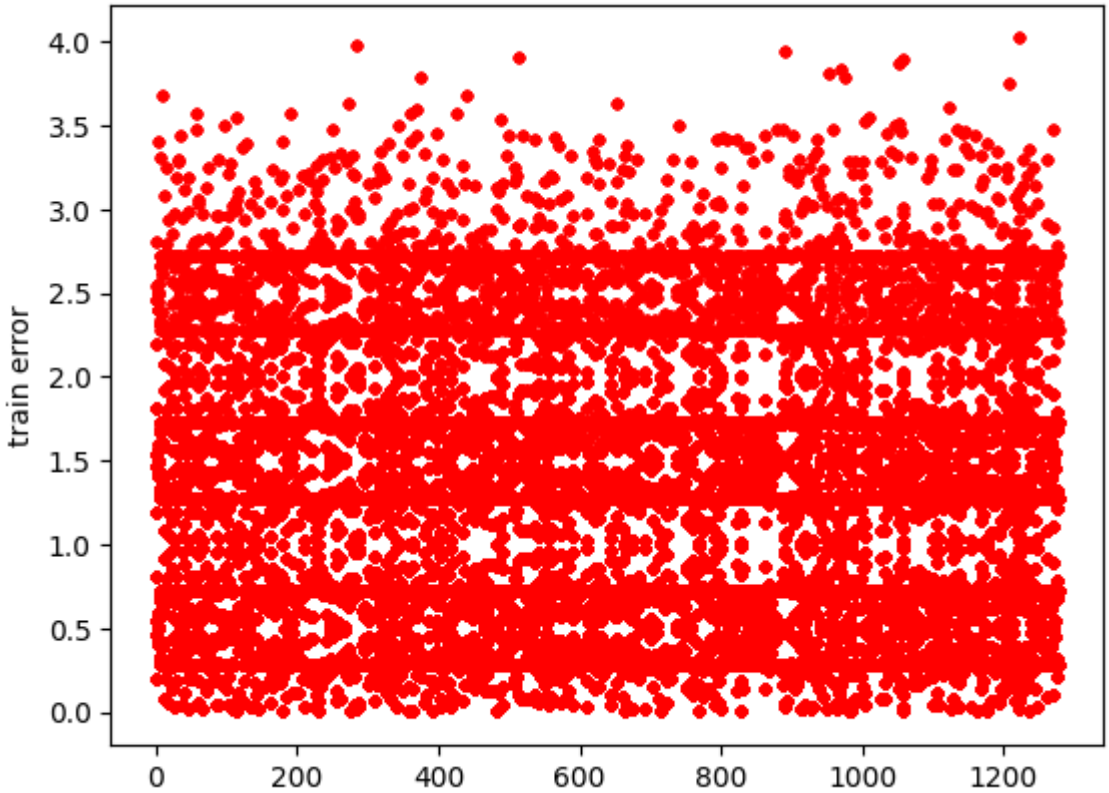
```
# df1.plot(kind='bar',figsize=(10,8))
# plt.grid(which='major', linestyle='-', linewidth='0.5', color='green')
# plt.grid(which='minor', linestyle=':', linewidth='0.5', color='black')
# plt.show()

# //Winston20260426 eveApple2100h.ipynb ;
# cannot see Colab.Gemini code edits in Chrome on Apple ;
df_train25.plot(kind='bar',figsize=(6,4.5))
plt.grid(which='major', linestyle='-', linewidth='0.5', color='green')
plt.minorticks_on()
plt.grid(which='minor', linestyle=':', linewidth='0.5', color='black')
# //Winston20260426 eveApple2100h.ipynb ;
plt.tight_layout()
plt.show()
```



In [11]:

```
plt.plot(train_error,'r.')
plt.ylabel('train error')
plt.show()
```



In [12]:

```
# testing the model with the test data
prediction = estimator.predict(X_test)
```

```
test_error =  np.abs(Y_test - prediction)
mean_error = np.mean(test_error)
min_error = np.min(test_error)
max_error = np.max(test_error)
std_error = np.std(test_error)
print('\nTesting results:\nmean=',mean_error,'\nmin=',min_error,'\nmax=',ma
```

```
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step

Testing results:
mean= 0.6863526630401612
min= 2.86102294921875e-06
max= 3.9832096099853516
std= 0.49790212552250485
```

In [13]:

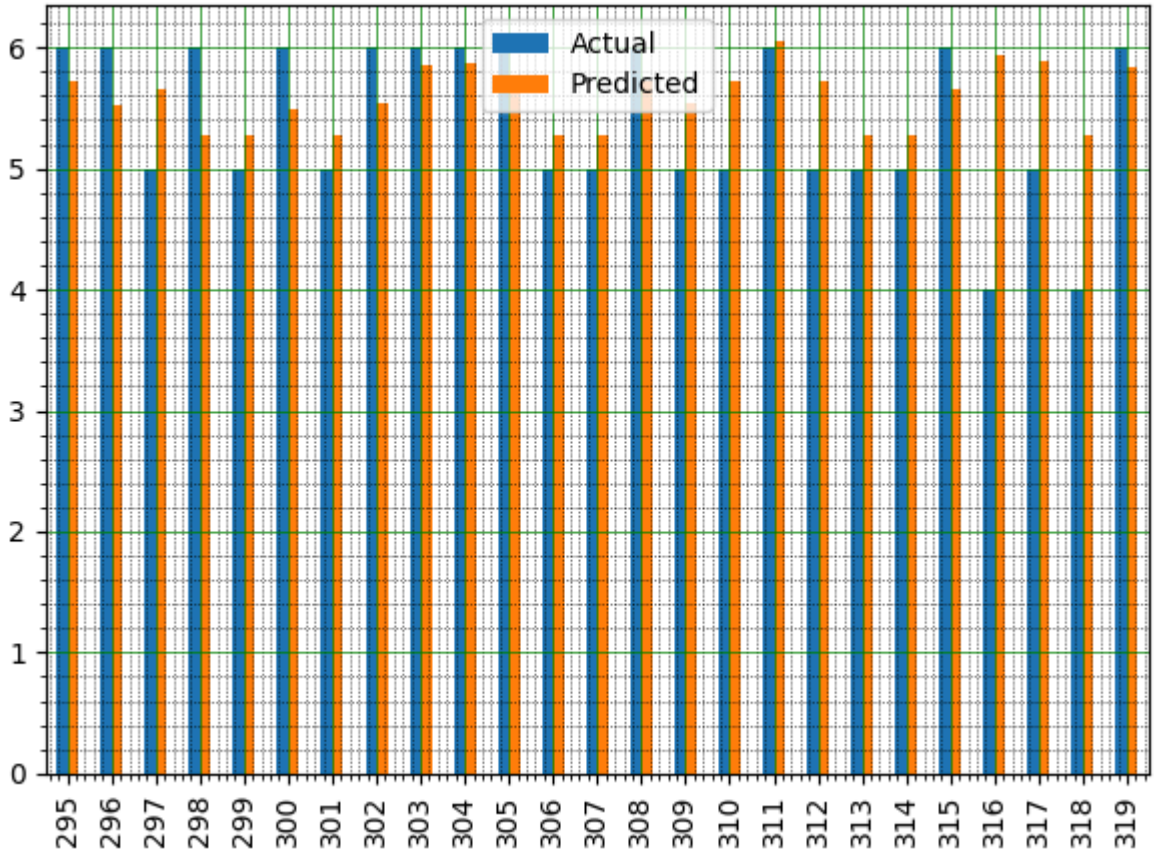
```
# and showing the results
df_test1 = pd.DataFrame({'Actual': Y_test, 'Predicted': prediction.ravel()
df_test25 = df_test1.tail(25)
print(df_test25)
```

```
     Actual  Predicted
295     6.0   5.727218
296     6.0   5.520314
297     5.0   5.648234
298     6.0   5.275720
299     5.0   5.275720
300     6.0   5.494576
301     5.0   5.275720
302     6.0   5.542541
303     6.0   5.858410
304     6.0   5.873368
305     6.0   5.656260
306     5.0   5.275720
307     5.0   5.275720
308     6.0   5.726296
309     5.0   5.538679
310     5.0   5.716629
311     6.0   6.048903
312     5.0   5.729024
313     5.0   5.275720
314     5.0   5.275720
315     6.0   5.663683
316     4.0   5.944163
317     5.0   5.884384
318     4.0   5.275720
319     6.0   5.842120
```

In [14]:

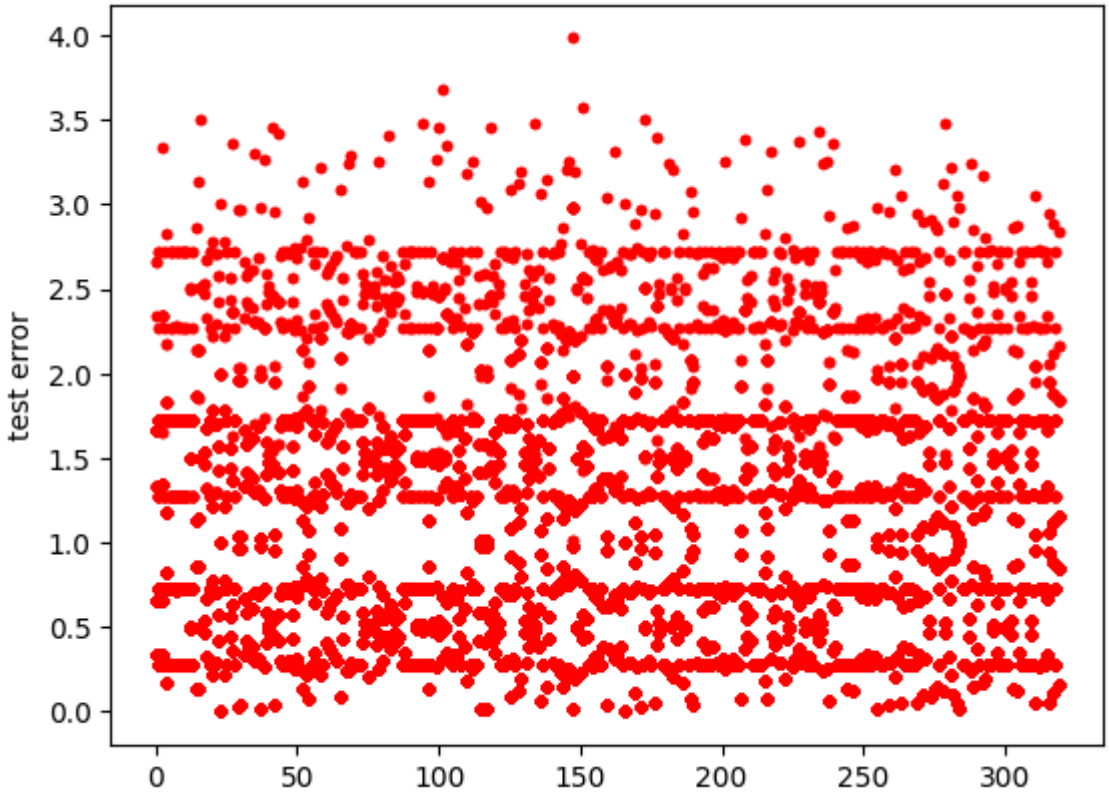
```
# df1.plot(kind='bar',figsize=(10,8))
# plt.grid(which='major', linestyle='-', linewidth='0.5', color='green')
# plt.grid(which='minor', linestyle=':', linewidth='0.5', color='black')
# plt.show()

# //Winston20260426 eveApple2100h.ipynb ;
# cannot see Colab.Gemini code edits in Chrome on Apple ;
df_test25.plot(kind='bar',figsize=(6,4.5))
plt.grid(which='major', linestyle='-', linewidth='0.5', color='green')
plt.minorticks_on()
plt.grid(which='minor', linestyle=':', linewidth='0.5', color='black')
# //Winston20260426 eveApple2100h.ipynb ;
plt.tight_layout()
plt.show()
```



In [15]:

```
plt.plot(test_error,'r.')
plt.ylabel('test error')
plt.show()
```



Practices

1. train the model for 1400 epochs. what are the training and testing MAE?
2. repeat that with 6000 training epochs. what are the training and testing MAE?
3. move back to 900 training epochs. add two convolutional and one dense layer with the same features of the layers used. what are the training and testing MAE?
4. go for 6000 training epochs. Do you feel any overfitting?
5. set back everything to the original setting, then try to normalize the training and testing features and outputs using standard_scaler. Then see if anything gets better or not. You may use insttructions like what you can see in the next cell.

points:

- use GPU as your runtime type
- if the notebook overflows, change the verbose to 0, e.g. verbose=0 in the training cell.

In [16]:

```
'''
# normalization
from sklearn.preprocessing import StandardScaler

# Create scalers
scaler_X = StandardScaler()
scaler_y = StandardScaler()

# Standardize X_train and X_test
X_train_scaled = scaler_X.fit_transform(X_train)
X_test_scaled = scaler_X.transform(X_test)

# Standardize Y_train and Y_test (only if y is continuous, e.g., in regress
y_train_scaled = scaler_y.fit_transform(Y_train.reshape(-1, 1))  # Reshape
y_test_scaled = scaler_y.transform(Y_test.reshape(-1, 1))
'''
```

Out[16]:

```
'\n# normalization\nfrom sklearn.preprocessing import StandardScaler\n\n# C
reate scalers\nscaler_X = StandardScaler()\nscaler_y = StandardScaler()\n\n
# Standardize X_train and X_test\nX_train_scaled = scaler_X.fit_transform(X
_train)\nX_test_scaled = scaler_X.transform(X_test)\n\n# Standardize Y_trai
n and Y_test (only if y is continuous, e.g., in regression)\ny_train_scaled
= scaler_y.fit_transform(Y_train.reshape(-1, 1))  # Reshape if needed\ny_te
st_scaled = scaler_y.transform(Y_test.reshape(-1, 1))\n'
```